Universidad Autónoma de Chiriquí
Facultad de Economía
Escuela de Ciencias de Computación

Lic. en Gestión de Tecnología de la Información

"Programación V – PROG 309"

Estudiantes:
Arian Cervantes
Jairo Chávez
Débora Cisneros
Daniela Pittí

Prof. Alexis Flores

Laboratorio – Semana 6

Especificación de requisitos de software (SRS)
Caso: Sistema de gestión de inventario en capas

II Semestre

1. Introducción

1.1 Propósito

Este documento especifica los requisitos de software del sistema de gestión de inventario que se desarrolla bajo el patrón de capas desde el Eje Temático N.º 1. Sirve como base para el diseño de la arquitectura en capas (Semana 7) y como referencia de verificación para las pruebas del sistema.

1.2 Alcance

El sistema permitirá a una pequeña o mediana empresa registrar y consultar productos, controlar sus existencias, registrar compras y ventas, y obtener reportes del valor del inventario. Queda fuera del alcance la facturación electrónica fiscal y la contabilidad general; el sistema se limita al control operativo del inventario.

1.3 Definiciones

Término	Definición
Producto	Artículo identificado por un código único (SKU), con nombre, categoría, precio de costo, precio de venta y existencia.
Existencia (stock)	Cantidad disponible de un producto en un momento dado.
Stock mínimo	Cantidad por debajo de la cual el producto se considera en nivel crítico.
Movimiento de inventario	Registro de entrada (compra, ajuste) o salida (venta, ajuste) que modifica la existencia.
Valor del inventario	Suma de existencia × precio de costo de todos los productos activos.
RF / RNF	Requisito funcional / Requisito no funcional.
Capas	Patrón arquitectónico que separa presentación, lógica de negocio y acceso a datos.

2. Requisitos funcionales

Cada requisito describe una sola función y puede verificarse ejecutándola. Se redactan con el formato “El sistema debe…”.

ID	Requisito	Verificación
RF-01	El sistema debe permitir registrar un producto con código (SKU), nombre, categoría, precio de costo, precio de venta y existencia inicial.	Registrar y ver el producto en el listado
RF-02	El sistema debe rechazar el registro de un producto cuyo SKU ya exista.	Intentar duplicar un SKU
RF-03	El sistema debe permitir modificar los datos de un producto existente.	Editar y consultar
RF-04	El sistema debe permitir desactivar un producto sin eliminar su historial de movimientos.	Desactivar y revisar historial

ID	Requisito	Verificación
RF-05	El sistema debe permitir buscar productos por nombre, SKU o categoría.	Buscar con cada criterio
RF-06	El sistema debe permitir registrar proveedores con nombre y datos de contacto.	Registrar y consultar
RF-07	El sistema debe permitir registrar una compra que aumente la existencia de los productos incluidos.	Comparar existencia antes y después
RF-08	El sistema debe permitir registrar una venta que disminuya la existencia de los productos vendidos.	Comparar existencia antes y después
RF-09	El sistema debe impedir registrar una venta cuando la cantidad solicitada supere la existencia disponible.	Vender más de lo disponible
RF-10	El sistema debe permitir registrar un ajuste manual de inventario indicando el motivo.	Ajustar y ver el motivo en el movimiento
RF-11	El sistema debe mostrar el historial de movimientos de un producto, con fecha, tipo, cantidad y usuario.	Consultar historial tras varios movimientos
RF-12	El sistema debe calcular el valor total del inventario.	Comparar con cálculo manual
RF-13	El sistema debe generar una alerta cuando la existencia de un producto sea menor o igual a su stock mínimo.	Reducir stock hasta el mínimo
RF-14	El sistema debe generar un reporte de ventas por rango de fechas.	Generar reporte con datos conocidos
RF-15	El sistema debe permitir exportar los reportes en formato PDF y CSV.	Exportar y abrir el archivo
RF-16	El sistema debe autenticar a los usuarios con usuario y contraseña antes de permitir el acceso.	Acceso con credenciales válidas e inválidas
RF-17	El sistema debe asignar a cada usuario un rol (administrador, vendedor, consulta) que limite las operaciones permitidas.	Probar una operación restringida con cada rol

3. Requisitos no funcionales

Los requisitos se redactan de forma medible. Se incluyen los tres tipos obligatorios (rendimiento, seguridad y mantenibilidad) y, adicionalmente, escalabilidad, usabilidad y confiabilidad.

ID	Tipo	Requisito medible	Medición
RNF-01	Rendimiento	La búsqueda de productos debe responder en menos de 1 segundo con un catálogo de hasta 50 000 productos.	Prueba de carga
RNF-02	Rendimiento	El registro de una venta de hasta 20 líneas debe completarse en menos de 2 segundos.	Medición en pruebas
RNF-03	Rendimiento	El sistema debe soportar 10 usuarios concurrentes con un tiempo de respuesta promedio menor a 2 segundos.	Prueba de concurrencia

ID	Tipo	Requisito medible	Medición
RNF-04	Seguridad	Las contraseñas deben almacenarse con hash y sal (por ejemplo, PBKDF2 o bcrypt); nunca en texto plano.	Revisión de la base de datos
RNF-05	Seguridad	La aplicación debe conectarse a MySQL con un usuario dedicado de privilegios mínimos (nunca root) y con contraseña, y el puerto de MySQL no debe quedar expuesto fuera de la red local.	Revisión de usuarios y permisos en MySQL
RNF-06	Seguridad	La sesión debe expirar tras 15 minutos de inactividad y la cuenta debe bloquearse tras 5 intentos fallidos consecutivos.	Prueba funcional
RNF-07	Seguridad	Toda operación debe validarse en la capa de negocio y usar consultas parametrizadas (PreparedStatement) para prevenir inyección SQL.	Revisión de código / pruebas
RNF-08	Mantenibilidad	El código debe organizarse en tres capas (presentación, negocio, datos) sin que la presentación acceda directamente a la base de datos.	Revisión de dependencias
RNF-09	Mantenibilidad	Las clases de la capa de negocio deben tener una cobertura de pruebas unitarias de al menos 70 %.	Reporte de cobertura
RNF-10	Mantenibilidad	Un cambio en una regla de negocio debe requerir modificar archivos de una sola capa.	Revisión de un cambio de ejemplo
RNF-11	Escalabilidad	El sistema debe permitir crecer a 500 000 movimientos de inventario sin que los tiempos de RNF-01 y RNF-02 aumenten más de 20 %.	Prueba con volumen
RNF-12	Usabilidad	Un usuario nuevo debe poder registrar una venta en menos de 3 minutos tras una capacitación de 15 minutos.	Prueba con usuarios
RNF-13	Confiabilidad	Debe realizarse un respaldo diario de la base de datos (mysqldump) y esta debe poder restaurarse en menos de 15 minutos.	Prueba de restauración

3.1 Impacto del requisito de rendimiento en la capa de datos

Para cumplir RNF-01 y RNF-02 (respuesta en menos de 1 y 2 segundos), el equipo discutió qué cambiaría en la capa de datos del proyecto:

- **Índices:** crear índices sobre SKU (único), nombre y categoría de producto, y sobre fecha y producto en la tabla de movimientos, para evitar búsquedas secuenciales.
- **Paginación:** paginar los resultados (por ejemplo, 50 registros por consulta) en lugar de cargar el catálogo completo.
- **Consultas eficientes:** traer solo las columnas necesarias y evitar consultas repetidas por cada fila (problema N+1) mediante consultas con uniones o carga anticipada.

- **Transacciones:** registrar cada venta o compra y la actualización de existencias en una sola transacción corta (motor InnoDB de MySQL, con commit y rollback desde JDBC), para mantener la consistencia sin bloquear la tabla por mucho tiempo.
- **Existencia precalculada:** mantener la existencia actual en la tabla de productos, y no recalcularla sumando todo el historial de movimientos en cada consulta.
- **Caché:** guardar en memoria datos de lectura frecuente y poco cambiantes (categorías, proveedores) para reducir viajes a la base de datos.

4. Lenguaje y herramientas

4.1 Tecnologías seleccionadas

Componente	Elección
Lenguaje	Java (JDK 17 LTS)
Entorno de desarrollo	Eclipse IDE for Java Developers
Base de datos	MySQL, ejecutado con XAMPP (administración con phpMyAdmin)
Acceso a datos	JDBC con MySQL Connector/J, aplicando el patrón DAO
Interfaz (presentación)	Java Swing (diseño visual con WindowBuilder de Eclipse)
Pruebas	JUnit 5
Gestión de dependencias	Maven
Control de versiones	Git y GitHub

Nota: XAMPP incluye Apache y MySQL (en sus versiones recientes, MariaDB, compatible con MySQL). En este proyecto se usa únicamente el servidor de base de datos y phpMyAdmin; la aplicación Java se ejecuta desde Eclipse y se conecta a la base mediante JDBC.

4.2 Justificación según los requisitos no funcionales

- **Rendimiento:** Java es un lenguaje compilado que se ejecuta sobre la JVM, con buen desempeño y soporte de hilos para atender varias operaciones a la vez (RNF-03). JDBC permite controlar las consultas, los índices y las transacciones discutidas en la sección 3.1, y MySQL con motor InnoDB maneja bien los volúmenes de RNF-01, RNF-02 y RNF-11 cuando las tablas están indexadas.
- **Seguridad:** JDBC con PreparedStatement evita la inyección SQL (RNF-07); existen bibliotecas maduras para el hash de contraseñas con sal, como BCrypt (RNF-04); y MySQL permite crear un usuario de base de datos con privilegios mínimos (RNF-05). Los roles y el bloqueo de cuenta (RF-16, RF-17, RNF-06) se implementan en la capa de negocio.

- **Mantenibilidad:** Java es un lenguaje orientado a objetos con tipado estático, lo que facilita separar el sistema en tres paquetes (presentación, negocio y datos) con dependencias en una sola dirección, como exigen RNF-08 y RNF-10. Las interfaces de Java permiten definir los DAO y reemplazar la implementación sin tocar la lógica de negocio, y JUnit permite alcanzar la cobertura de RNF-09.
- **Viabilidad para el equipo:** Eclipse, XAMPP y MySQL son gratuitos, se instalan fácilmente en los equipos del laboratorio y permiten al equipo desarrollar y probar sin costos de infraestructura. XAMPP facilita crear, respaldar (RNF-13) y administrar la base de datos desde phpMyAdmin.

Alternativa considerada: una aplicación web con Servlets/JSP sobre Apache Tomcat. Se descartó para esta etapa porque requiere configurar un servidor adicional a XAMPP; además, al mantener la arquitectura en capas, la presentación en Swing podría sustituirse más adelante por una interfaz web sin modificar las capas de negocio y de datos.

5. Conclusiones

- Los requisitos funcionales definen qué debe hacer cada capa: la presentación recibe y muestra datos (RF-05, RF-11, RF-15), la capa de negocio concentra las reglas (RF-02, RF-09, RF-12, RF-13, RF-17) y la capa de datos persiste productos y movimientos (RF-01, RF-07, RF-08).
- El requisito de rendimiento influye directamente en la capa de datos (índices, paginación, transacciones cortas y existencia precalculada), por lo que el diseño de las tablas de la Semana 7 deberá contemplarlos desde el inicio.
- Los requisitos de seguridad exigen que las reglas y validaciones residan en la capa de negocio y no solo en la interfaz, de modo que ninguna operación pueda saltarse los controles de rol y de existencia.
- El requisito de mantenibilidad obliga a que las dependencias entre capas sean unidireccionales (Presentación → Negocio → Datos), lo que se reflejará en el diagrama de componentes y en la estructura de proyectos de la solución.
- La elección de Java, Eclipse y MySQL con XAMPP es coherente con los requisitos no funcionales medibles y sienta la base técnica para desarrollar la arquitectura en capas en la Semana 7.

6. Referencias

Oracle. (s. f.). Java Platform, Standard Edition documentation. https://docs.oracle.com/en/java/javase/

Oracle. (s. f.). MySQL reference manual. https://dev.mysql.com/doc/